

# Especificação Técnica de Requisitos: Plataforma de Gestão e Agendamento

Este documento descreve detalhadamente o escopo, as regras de negócio e as decisões arquiteturais para o desenvolvimento da plataforma de barbearia. A arquitetura baseia-se em **Next.js**, **PostgreSQL** e **Prisma ORM**, adotando princípios de **Clean Architecture** e **SOLID** para garantir escalabilidade, manutenibilidade e isolamento de responsabilidades entre *Controllers*, *UseCases* e *Repositories*.

## 1. Visão Geral da Arquitetura e Padrões

- Padrão de Projeto:** Implementação estrita da separação de preocupações. O framework (Next.js) atua apenas na camada de entrega (roteamento e UI). A lógica de negócios central reside em *UseCases* que são completamente agnósticos ao framework web, interagindo com o banco de dados exclusivamente através do padrão *Repository*.
- Infraestrutura Local:** Recomendada a containerização do banco de dados PostgreSQL via Docker (docker-compose) para simular o ambiente de produção localmente com precisão.
- Gerenciamento de Estado UI:** Para orquestrar o fluxo de *Lazy Registration*, sugere-se a utilização do *Zustand* ou da *Context API* nativa do React para reter as seleções de serviço e horário na memória do cliente até a consolidação da persistência.

## 2. Requisitos Funcionais Detalhados (RF)

Detalhamento das funcionalidades, incluindo regras de negócio (RN) e fluxos de exceção.

| ID / Módulo | Descrição e Regras de Negócio (RN)                                                                        |
|-------------|-----------------------------------------------------------------------------------------------------------|
| RF01        | <b>Descrição:</b> O sistema não permite a criação orgânica ou via interface de usuários com a role ADMIN. |

| ID / Módulo                                                          | Descrição e Regras de Negócio (RN)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Controle de Acesso Master                                            | <p><b>RN01.1:</b> O usuário Super Admin inicial deve ser gerado exclusivamente através de um script de <i>Seed</i> do Prisma durante o provisionamento do banco de dados.</p> <p><b>RN01.2:</b> Qualquer payload recebido em rotas públicas contendo role: "ADMIN" deve ser sumariamente ignorado e sanitizado no nível do Controller (via validação Zod).</p>                                                                                                                                                                                                                                                   |
| <b>RF02</b><br><br>Gestão de Funcionários                            | <p><b>Descrição:</b> Apenas perfis autenticados como ADMIN podem registrar novos BARBERS no sistema.</p> <p><b>RN02.1:</b> O Admin deve fornecer nome, e-mail e definir uma senha provisória, além de configurar os dias da semana e a carga horária de trabalho do novo funcionário.</p> <p><b>RN02.2:</b> O sistema deve enviar um e-mail ou notificação ao novo barbeiro para que ele redefina sua senha no primeiro login.</p>                                                                                                                                                                               |
| <b>RF03 &amp; RF04</b><br><br>Jornada do Cliente (Lazy Registration) | <p><b>Descrição:</b> O sistema deve priorizar a conversão permitindo navegação anônima até o ato da reserva.</p> <p><b>Fluxo:</b> O cliente acessa -&gt; Seleciona o Serviço -&gt; Seleciona o Barbeiro -&gt; Escolhe Data e Hora -&gt; Clica em "Confirmar".</p> <p><b>RN03.1:</b> Somente ao clicar em confirmar, o sistema exibe um modal ou redireciona para a tela de Auth (Login/Cadastro).</p> <p><b>RN03.2:</b> Após a autenticação bem-sucedida, o estado salvo na interface deve ser automaticamente submetido à API de agendamento POST /appointments, sem exigir que o cliente refaça os passos.</p> |
| <b>RF06</b><br><br>Motor de Agendamento                              | <p><b>Descrição:</b> O núcleo da aplicação, garantindo transparência de horários e prevenindo sobreposições em duas etapas.</p> <p><b>RN06.1 (Consulta de Disponibilidade - Frontend):</b> O sistema deve fornecer uma rota (ex: GET /api/barbers/{id}/availability) para retornar os horários disponíveis.</p> <p><b>RN06.2 (Geração e Cálculo de Blocos):</b> O UseCase gera a grade diária em blocos fixos de 30 minutos (ex: 09:00, 09:30). Ele calcula quantos blocos o serviço escolhido</p>                                                                                                               |

| ID / Módulo                                                           | Descrição e Regras de Negócio (RN)                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                       | <p>consume (ex: 50 min arredonda para 60 min = 2 blocos) e remove da grade os horários que conflitam com agendamentos PENDING ou COMPLETED.</p> <p><b>RN06.3 (Rede de Segurança - Backend):</b> No momento do POST /appointments, o CreateAppointmentUseCase realiza uma última checagem transacional com o banco de dados. Se os blocos de horário não estiverem mais disponíveis (concorrência real), aborta a transação com erro 409 Conflict.</p>               |
| <b>RF07 &amp; RF08</b><br><br>Máquina de Estados e Painel do Barbeiro | <p><b>Descrição:</b> O barbeiro gerencia sua própria fila de trabalho diariamente.</p> <p><b>RN07.1:</b> O status padrão ao criar é PENDING.</p> <p><b>RN07.2:</b> O barbeiro pode alterar o status para COMPLETED, CANCELED ou NO_SHOW (Não compareceu).</p> <p><b>RN07.3:</b> Uma vez marcado como COMPLETED, o registro torna-se imutável financeiramente pelo barbeiro (apenas o Admin pode auditar e corrigir).</p>                                            |
| <b>RF09 a RF11</b><br><br>Financeiro e Comissionamento                | <p><b>Descrição:</b> Consolidação de métricas financeiras para o negócio.</p> <p><b>RN09.1:</b> Apenas agendamentos COMPLETED geram valor financeiro positivo nas consultas agregadas do dashboard.</p> <p><b>RN09.2:</b> A tabela de serviços deve conter um campo commission_rate (ex: 0.40 para 40%).</p> <p><b>RN09.3:</b> O Dashboard deve exibir: Receita Bruta Total, Valor Líquido (Barbearia) e Repasse Devido (para cada Barbeiro) de forma agregada.</p> |

### 3. Requisitos Não Funcionais Detalhados (RNF)

---

| ID / Categoria                               | Especificação Técnica                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RNF01</b><br><br>Stack Base               | <p><b>Next.js (App Router):</b> Para SSG/SSR otimizando SEO do catálogo de serviços e Server Actions para mutações de dados mais ágeis.</p> <p><b>Prisma ORM:</b> Tipagem forte end-to-end e facilidade na construção das queries de agregação financeiras.</p> <p><b>PostgreSQL:</b> Banco robusto para controle de concorrência e constraints de chave estrangeira.</p> |
| <b>RNF03</b><br><br>Segurança e Sessão       | <p>As senhas devem ser hashadas com bcryptjs. A autenticação utilizará <b>Cookies HttpOnly</b> (via NextAuth/Auth.js) para evitar ataques de XSS e interceptação de tokens no localStorage. Middlewares do Next.js interceptarão rotas de /admin e /dashboard, validando o token JWT e a role do usuário antes de renderizar a página ou permitir o hit na API.</p>       |
| <b>RNF05</b><br><br>Concorrência e Banco     | <p>Utilização de transações no banco de dados para evitar condições de corrida (Race Conditions). Se dois clientes tentarem marcar as 15:00 ao exato mesmo milissegundo, o isolamento transacional do Postgres garantirá que apenas o primeiro obtenha o lock do recurso.</p>                                                                                             |
| <b>RNF07</b><br><br>Performance de Consultas | <p>Para a construção do dashboard financeiro, em vez de carregar todos os objetos de agendamento para a memória do Node.js, utilizar as funções nativas prisma.appointment.aggregate() e groupBy, delegando o processamento matemático (SUM, AVG) diretamente ao motor do PostgreSQL.</p>                                                                                 |

## 4. Integrações Futuras (Roadmap)

A integração de notificações via WhatsApp (RF12) demandará o provisionamento de um serviço auxiliar de CRON Jobs (como GitHub Actions, Vercel Cron ou um worker isolado) que faça polling constante no banco para identificar agendamentos cuja coluna start\_time esteja a 1 hora do tempo atual. Ao identificar, deve disparar a carga útil para provedores de mensageria oficiais (considerando as exigências e limites de linhas da Meta para contas empresariais

verificadas) ou APIs não oficiais (Z-API, Evolution API), entregando o lembrete ao cliente.